Técnicas de Programação II 2026

Struct

FELIPE CARVALHO PELLISON

FELIPE.CARVALHO@BARAODEMAUA.BR

Introdução

Tipos de dados

Tipos de dados em linguagens de programação é o conjunto de valores que uma determinada variável pode assumir

Os tipos de dados são divididos em:

Tipos **primitivos** de dados;

Tipos **estruturados** de dados;

Introdução

Tipos primitivos de dados:

Os tipos primitivos de dados são os tipos básicos da linguagem de programação (int, char, float, bool, etc)

Tipos estruturados de dados:

Os tipos estruturados de dados são construídos a partir dos tipos primitivos e são exemplos de estrutura de dados (vetores, struct, unions)

Introdução

As estruturas de dados de se dividem em:

Homogêneos (vetores, matrizes e registros do mesmo tipo de dado)

```
struct Coordenadas {
int ponto_X;
int ponto_Y;
};
```

Heterogêneos (registros com tipos diferente).

```
struct Funcionario{
int codFunc;
string nome[30];
float salario;
};
```

Estruturas

Estrutura

Estrutura é uma coleção de variáveis de diferentes tipos de dados sob um único nome.

A palavra-chave **struct** define um tipo de estrutura seguido por um identificador (nome da estrutura).

Dentro da estrutura declara-se um ou mais membros (declarar variáveis dentro de chaves) dessa estrutura.

Definindo Estruturas

```
1   #include <iostream>
2   #include <string>
3   using namespace std;
4
5   struct Pessoa{
6       string nome;
7       int idade;
8       float salario;
9   };
```

Cada campo (variável) da estrutura pode ser acessada usando o operador "."(ponto)

Inicializando e exibindo estruturas

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Pessoa{
    string nome;
    int idade;
    float salario;
};
```

```cpp
int main (){
    Pessoa aluno1;

    cout << "Informe o nome do aluno: ";
    getline(cin, aluno1.nome);

    cout << "Informe a idade: ";
    cin >> aluno1.idade;

    cout << "Informe o Salario: ";
    cin >> aluno1.salario;

    cout << endl << endl << "Exibindo informacoes." << endl;
    cout << "Nome: " << aluno1.nome << endl;
    cout << "Idade: " << aluno1.idade << endl;
    cout << "Salario: " << aluno1.salario << endl;

    return 0;
}
```

Inicializando e exibindo estruturas

```
11  int main (){
12      Pessoa aluno1 = {"Joao Pedro", 20, 1000.00};
13
14      cout << endl << endl << "Exibindo informacoes." << endl;
15      cout << "Nome: " << aluno1.nome << endl;
16      cout << "Idade: " << aluno1.idade << endl;
17      cout << "Salario: " << aluno1.salario << endl;
18
19      return 0;
20  }
21
```

Inicializando e exibindo estruturas

- S1 + S 2;
- S1 - S 2;
- S1 * S 2;
- S1 / S 2;

- S1 = S 2;

Inicializando e exibindo estruturas

```
struct Pessoa
{
  string nome;
  int idade;
  float salario;
};

int main() {

  Pessoa aluno1 = {"João Pedro", 20, 1000.00};
  Pessoa aluno2;

  aluno2 = aluno1;

  cout << endl << endl << "Exibindo informações." << endl;
  cout << "Nome: " << aluno2.nome << endl;
  cout << "Idade: " << aluno2.idade << endl;
  cout << "Salário: " << aluno2.salario << endl << endl;

  system("pause");
  return 0;
}
```

Estruturas como parâmetro

```
void exibirDados(Pessoa paramAluno);

int main() {

  Pessoa aluno1;

  cout << "Entre com o nome do aluno: ";
  getline(cin, aluno1.nome);

  cout << "Entre com a idade: ";
  cin >> aluno1.idade;

  cout << "Entre com o salário: ";
  cin >> aluno1.salario;

  exibirDados(aluno1);

  system("pause");
  return 0;
}
```

```
void exibirDados(Pessoa paramAluno) {

  cout << endl << endl << "Exibindo informações." << endl;
  cout << "Nome: " << paramAluno.nome << endl;
  cout << "Idade: " << paramAluno.idade << endl;
  cout << "Salário: " << paramAluno.salario << endl << endl;

}
```

Estruturas como retorno

```
Pessoa obterDados() {
  Pessoa aluno;

  cout << "Entre com o nome do aluno: ";
  getline(cin, aluno.nome);

  cout << "Entre com a idade: ";
  cin >> aluno.idade;

  cout << "Entre com o salário: ";
  cin >> aluno.salario;

  return aluno;
}

void exibirDados(Pessoa paramAluno) {

  cout << endl << endl << "Exibindo informações." << endl;
  cout << "Nome: " << paramAluno.nome << endl;
  cout << "Idade: " << paramAluno.idade << endl;
  cout << "Salário: " << paramAluno.salario << endl << endl;
}
```

```
Pessoa obterDados();
void exibirDados(Pessoa paramAluno);

int main() {

  Pessoa aluno1;

  aluno1 = obterDados();

  exibirDados(aluno1);

  system("pause");
  return 0;
}
```

Typedef

A linguagem C++ permite que o programador defina os seus próprios tipos com base em outros tipos de dados existentes.

Para isso, utiliza-se o comando ***typedef***, cuja forma geral é:

typedef **tipo_existente** novo_nome;

Exemplo

```
296 #include <iostream>
297 #include <stdlib.h>
298 #include <string>
299 using namespace std;
300
301 typedef int inteiro;
302
303 int main() {
304
305     int x = 10;
306     inteiro y = 20;
307     int z = x + y;
308     cout << "a soma é = " << z << endl;
309
310     return 0;
311 }
```

Typedef

O **typedef** é muito utilizado para definir nomes mais simples para estrutura, evitando carregar a palavra **struct** sempre que referenciamos a estrutura

```
296 #include <iostream>
297 #include <stdlib.h>
298 #include <string>
299 using namespace std;
300
301 struct Produto{
302     int CodProduto;
303     string descricao;
304 };
305
306 typedef struct Produto prod;
307
308 int main() {
309
310     // são equivalentes
311     struct Produto p1;
312     prod p2;
313
314     return 0;
315 }
```

Estruturas aninhadas

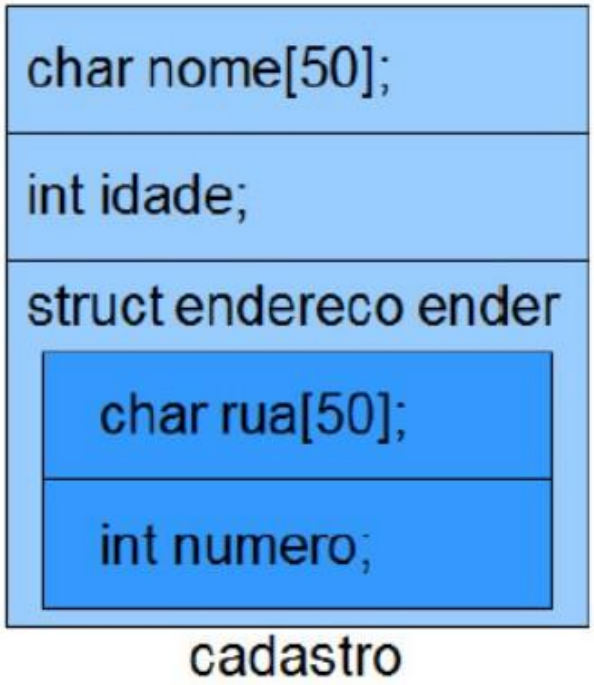
Podemos declarar uma estrutura que possua uma variável do tipo de outra estrutura **previamente** definida.

A uma estrutura que contenha outra estrutura dentro dela damos o nome de estruturas aninhadas.

Estruturas aninhadas

Exemplo: struct aninhada.

```
1 struct endereco{
2   char rua[50]
3   int numero;
4 };
5 struct cadastro{
6   char nome[50];
7   int idade;
8   struct endereco
        ender;
9 };
```



Estruturas aninhadas

```
struct Endereco {
  string rua;
  int nro;
};
struct Pessoa
{
  string nome;
  int idade;
  float salario;
  Endereco endereco;
};
int main() {

  Pessoa aluno1 = { "João Pedro", 20, 1000.00, {"Amapa", 35} };

  cout << endl << endl << "Exibindo informações." << endl;
  cout << "Nome: " << aluno1.nome << endl;
  cout << "Idade: " << aluno1.idade << endl;
  cout << "Salário: " << aluno1.salario << endl;
  cout << "Endereço: " << aluno1.endereco.rua << ", " <<  aluno1.endereco.nro << endl << endl;

  system("pause");
  return 0;
}
```

Vetor de estruturas

SOLUÇÃO: criar um **array de estruturas**.

Sua declaração é similar a declaração de um array de um tipo básico

```
struct Produto Prod[10];
```

Desse modo, declara-se um **array de 10 posições**, onde cada posição é do tipo **struct Produto**.

Vetor de estruturas

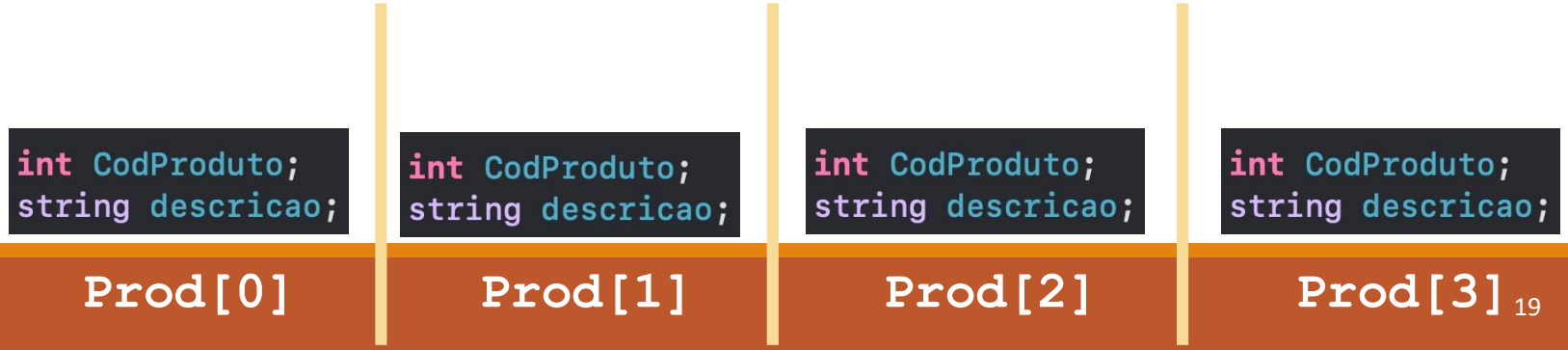
Lembrando:

struct: define um “conjunto” de variáveis que podem ser de tipos diferentes;

vetor: é uma “lista” de elementos de mesmo tipo.

```
234 struct Produto
235 {
236     int CodProduto;
237     string descricao;
238 };
```

```
struct Produto Prod[10];
```



Vetor de estruturas

Em um vetor de estruturas, o operador de ponto (**.**) vem depois dos colchetes (**[ ]**) do índice do **array**.

```
268 int main() {
269
270     Produto Prod[2];
271
272     cout << "Entre com código do produto 1: ";
273     cin >> Prod[0].CodProduto;
274
275     cout << "Entre com a descrição do produto 1: ";
276     cin.ignore();
277     getline(cin, Prod[0].descricao);
278
279     cout << "Entre com código do produto 2: ";
280     cin >> Prod[1].CodProduto;
281
282     cout << "Entre com a descrição do produto 2: ";
283     cin.ignore();
284     getline(cin, Prod[1].descricao);
```

Vetor de estruturas

cin armazena no buffer tudo que o usuário digitou, inclusive o ENTER (\n)

Mas, ele atribui na variável tudo o que foi digitado, **menos** o ENTER (\n)
O ENTER (\n) fica **sobrando** no buffer

Quando usamos o **getline** **<u>DEPOIS</u>** **de um cin**, precisamos usar o comando: **cin.ignore()**;

Para ele **ignorar** o ENTER (\n) que tinha ficado no buffer

Vetor de estruturas

```cpp
#include <iostream>
#include <stdlib.h>
#include <string>

using namespace std;

struct Produto
{
    int CodProduto;
    string descricao;
};
```

```cpp
int main() {

    Produto Prod[2];

    cout << "Entre com código do produto 1: ";
    cin >> Prod[0].CodProduto;

    cout << "Entre com a descrição do produto 1: ";
    cin.ignore();
    getline(cin, Prod[0].descricao);

    cout << "Entre com código do produto 2: ";
    cin >> Prod[1].CodProduto;

    cout << "Entre com a descrição do produto 2: ";
    cin.ignore();
    getline(cin, Prod[1].descricao);
```

```cpp
    cout << endl << endl << "Exibindo informações." << endl;
    cout << "========== Produto 1 ============" << endl;
    cout << "Código: " << Prod[0].CodProduto << endl;
    cout << "Descrição: " << Prod[0].descricao << endl << endl;

    cout << "========== Produto 2 ============" << endl;
    cout << "Código: " << Prod[1].CodProduto << endl;
    cout << "Descrição: " << Prod[1].descricao << endl << endl;

    system("pause");
    return 0;
}
```